大模型推理全链路组件间通信与协同体系深度解析（以 vLLM 为例）

目录

- 1. 引言与全局通信架构全景图
- 2. 软件层：内部模块级通信机制
  - 2.1 API Server 与 AsyncLLMEngine 的异步事件通信
  - 2.2 Tokenizer 与 Detokenizer 的流水线流式通信
  - 2.3 AsyncLLMEngine 与 EngineCore / Scheduler 的解耦通信
  - 2.4 调度器与显存块管理器（BlockManager）的状态交互
  - 2.5 Engine 与 Worker 之间的多进程通信（Ray / Multiprocessing / ZMQ / IPC）
- 3. 软硬件接口层通信：Host 与 Device 协同
  - 3.1 PCIe / CXL 总线协议与物理链路
  - 3.2 内存映射 I/O (MMIO) 与 GPU Doorbell 敲铃机制
  - 3.3 锁页内存（Pinned Host Memory）与异步 DMA 传输
  - 3.4 CUDA Stream、Event 与 CUDA Graph 拓扑消除 Host 开销
- 4. 单卡微架构层：GPU 内部硬件模块通信
  - 4.1 SM 内部：Warp 调度器、寄存器堆与 Tensor Core 间传输
  - 4.2 SM 与 Shared Memory (SRAM) / L1 Cache 访存网络
  - 4.3 异步拷贝与硬件加速器（NVIDIA TMA 异步张量传输）
  - 4.4 Crossbar 片上互连网络与 L2 Cache 交互
  - 4.5 L2 Cache 到 HBM3/HBM3e 的 CoWoS 宽总线访存通信
  - 4.6 PagedAttention 算子的微架构寻址与通信放大剖析
- 5. 单机多卡层：Scale-Up 节点内硬件互连通信
  - 5.1 PCIe P2P 与 PCIe 交换芯片（PLX）通信
  - 5.2 NVLink 与 NVSwitch 全互联拓扑
  - 5.3 GPUDirect P2P 与 CUDA IPC 共享显存句柄
- 6. 跨机集群层：Scale-Out 节点间硬件互连通信
  - 6.1 网络基础设施：InfiniBand 与 RoCE v2 无损以太网
  - 6.2 GPUDirect RDMA (GDR) 零拷贝技术
  - 6.3 NCCL 集合通信原语的底层拓扑与算法（Ring vs Tree vs NVLS）
  - 6.4 网内计算硬件加速（In-Network Computing / SHARP）
- 7. 分布式并行策略与通信模式剖析
  - 7.1 张量并行（Tensor Parallelism, TP）通信时机与开销
  - 7.2 流水线并行（Pipeline Parallelism, PP）点对点通信

    - 7.3 专家并行（Expert Parallelism, EP / MoE）的 All-to-All 路由通信
    - 7.4 序列并行（Sequence Parallelism, SP / Context Parallelism, CP）通信
    - 7.5 投机推理（Speculative Decoding）Draft-Target 协同通信
- 8. Prefill 与 Decode 解耦架构（PD 分离）跨节点通信
    - 8.1 PD 分离的核心痛点与架构演进
    - 8.2 KV Cache 跨机远端传输协议与 GDR 写入机制
    - 8.3 元数据协同（Sequence ID、Block Table 映射与流控）
- 9. 统一调度方案与迭代管理机制
    - 9.1 连续批处理（Continuous Batching / Iteration-level Scheduling）
    - 9.2 分块预填充（Chunked Prefill）与时延平准通信调度
    - 9.3 调度决策闭环与跨模块反馈时延
- 10. 模型与显存动态加载、卸载与换入换出策略
    - 10.1 模型权重启动加载与并行广播分发
    - 10.2 动态多租户 LoRA 适配器热插拔与层级加载
    - 10.3 KV Cache 显存换入换出（Swapping）与抢占（Preemption）机制
    - 10.4 分级存储卸载架构（HBM -> Host DRAM -> Local NVMe -> Remote Object Store）
- 11. 软硬件通信层级性能矩阵与系统级优化全景
- 12. 总结与未来趋势展望

---

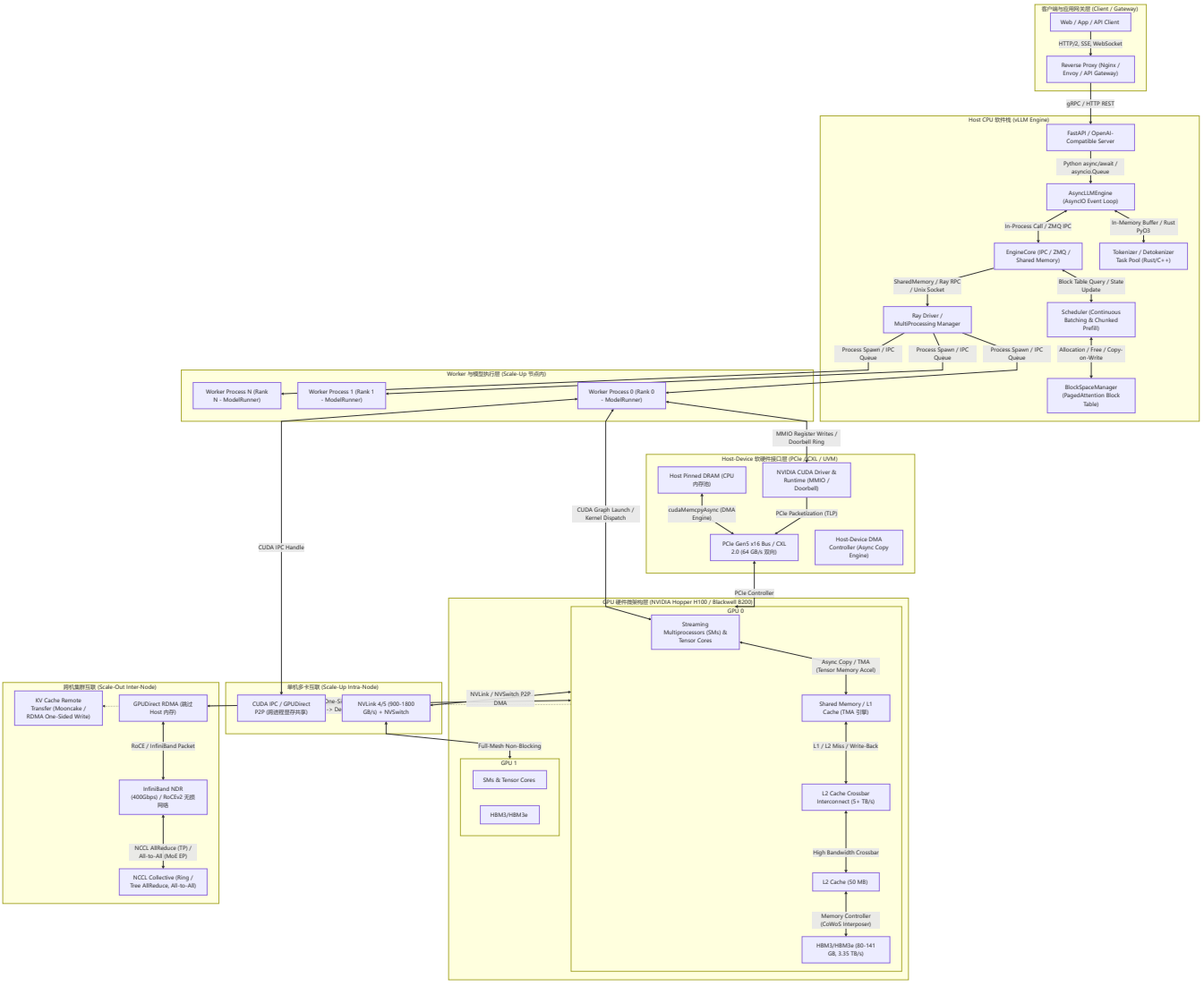
1. 引言与全局通信架构全景图

在大语言模型（LLM）推理系统中，吞吐量（Throughput）与延迟（首字延迟 TTFT 以及单 Token 解码延迟 ITL）的物理极限不仅取决于矩阵乘法的算力（FLOPs），更由**各个软硬件层级组件之间的通信带宽与通信往返时延（RTT）**决定。

作为工业界与开源领域最前沿的高性能推理引擎代表，**vLLM** 在软件架构设计（从经典 V0 架构到最新基于解耦通信的 V1/V2 架构）与底层硬件系统的协同中，构建了一套极为精密的通信拓扑。

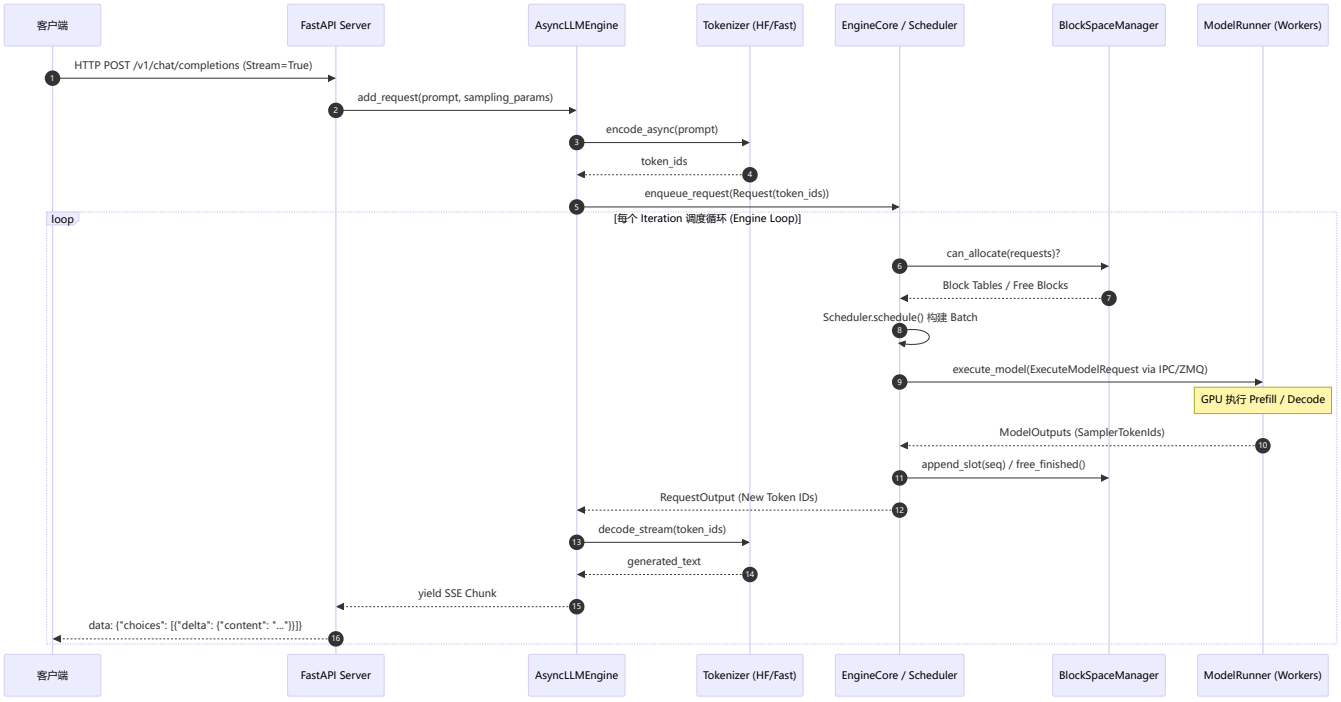
在大模型推理的生命周期中，一次请求的处理需要经历：客户端网关 -> 异步引擎事件循环 -> 批处理调度器 -> Worker 多进程 -> Host-Device 接口（PCIe/CXL）-> GPU 芯片内部存储互连网络（Crossbar/HBM/SRAM）-> 节点内 NVLink/NVSwitch 以及跨节点 InfiniBand/RoCE 远程直接显存访问（GPUDirect RDMA）。

全链路通信架构全景图



2. 软件层：内部模块级通信机制

vLLM 在软件层面为了规避 Python GIL（全局解释器锁）的执行瓶颈，最大化异步流水吞吐，并将单步调度的延迟控制在百微秒以内，实现了一套高性能的多进程事件驱动通信体系。



2.1 API Server 与 AsyncLLMEngine 的异步事件通信

1. **网络协议与事件循环**：
    - 基于 `FastAPI` 与 `Uvicorn`，工作在异步 `asyncio` 事件循环内。
    - 客户端长连接通过 **SSE（Server-Sent Events）** 或 **WebSocket** 建立，支持 Token 级流式回传。
2. **异步队列与推拉协同**：
    - 外部请求通过 `AsyncLLMEngine.add_request` 接入；
    - 引擎为每个 Request 创建独立的 `AsyncStream` 实体，其内部封装了一个 `asyncio.Queue`；
    - 引擎循环在产生新的 Token 时将其推入（Push）队列，API 层的异步生成器通过 `await queue.get()` 拉取（Pull）并格式化输出，实现非阻塞的背压流控（Backpressure）。

2.2 Tokenizer 与 Detokenizer 的流水线流式通信

1. **CPU 开销隔离**：
    - Tokenization（分词）与 Detokenization（反分词）是典型的 CPU 密集型操作。若在主引擎线程中执行，当 Prompt 达到万级别或并发极大时，会剧烈阻塞 GPU 调度节拍。
    - vLLM 采用底层基于 Rust/C++ 的 HuggingFace Tokenizer，在底层直接释放 Python GIL，或通过独立进程池/线程池异步执行。
2. **增量增量解码状态通信（Incremental Detokenizer）**：
    - 生成式场景下一个完整的 UTF-8 字符（如中文或 Emoji）可能跨越多个 Token；
    - Detokenizer 模块维护轻量级的未完结字节缓存状态，每个 Token 生成后仅执行增量字节解码（Incremental Decoding），避免重复全序列解码的巨大算力与内存复制损耗。

2.3 AsyncLLMEngine 与 EngineCore / Scheduler 的解耦通信

在 vLLM V1 架构演进中：

1. **解耦式核心架构**：
    - 负责网络 I/O、流式组织、Python 业务逻辑的前端 `AsyncLLMEngine`，与负责微秒级推理调度、显存分配与 GPU 指令分发的 `EngineCore` 被彻底剥离为独立进程。
2. **通信通道与零拷贝传输**：
    - **ZeroMQ (ZMQ) / Unix Domain Sockets (UDS)**：用于传递轻量级控制指令（如注册请求、终止信号）；
    - **共享内存（POSIX Shared Memory）**：针对包含成千上万 Token IDs 的大 Prompt，直接将其写入共享内存环形缓冲区，IPC 仅投递共享内存的偏移指针与长度，实现跨进程**零拷贝（Zero-Copy）传递。**

2.4 调度器与显存块管理器（BlockManager）的状态交互

1. **Block Table 映射查表**：
    - `Scheduler` 在每个推理步（Iteration）前调用 `BlockSpaceManager.can_allocate(seq_group)`；
    - `BlockSpaceManager` 查询物理空闲块池（Free Blocks List）以及前缀缓存树（RadixTree / Prefix Caching），返回新分配的物理 Block ID 列表或重用已有的前缀块。
2. **写时复制（Copy-on-Write, CoW）通信**：
    - 在多分支采样（Parallel Sampling / Beam Search）时，不同分支共享父序列的历史物理块，此时物理块引用计数增加。
    - 当某分支写入新 Token 触发分叉时，`BlockSpaceManager` 下发物理拷贝指令，向执行引擎 Worker 的执行队列中注入跨 Block 的显存拷贝任务。

2.5 Engine 与 Worker 之间的多进程通信（Ray / Multiprocessing / ZMQ / IPC）

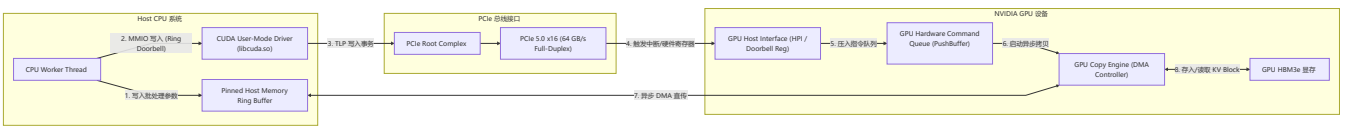
当启用多卡并行时（TP > 1 或 PP > 1），系统以多 Worker 模式启动：

1. **控制命令分发**：
    - **Ray 模式**：通过 Ray Actor 的 gRPC 协议进行异步 RPC 调度（`worker.execute_model.remote()`）；
    - **原生多进程模式（Native Multiprocessing）**：Rank 0 作为主控，通过 Unix 域套接字或命名管道向各子进程 Worker 广播 `ExecuteModelRequest`。
2. **C++ 绑定的 Broadcast Ring Buffer 极致加速**：
    - 为了消除 Python 对象的 Pickle 序列化延迟（通常每次数十微秒），vLLM V1 引入了底层 C++ 编写的广播环形队列，将调度元数据以平坦内存连续排布，其余 Rank 通过无锁自旋（Spin-wait）或 Eventfd 在 **< 15μs** 内感知并解析出批次信息。

---

3. 软硬件接口层通信：Host 与 Device 协同

CPU 负责统筹逻辑与构建批次，GPU 负责大规模并行张量计算。Host-Device 之间的交互带宽、协议开销与发射延迟，直接决定了模型在小 Batch 下是否会发生“CPU-Bound”。



3.1 PCIe / CXL 总线协议与物理链路

1. **物理拓扑与带宽**：
    - 当前主流服务器标配 **PCIe 5.0 x16**（单向 32 GB/s，双向全双工 64 GB/s）或正在演进的 **CXL 2.0/3.0**。
    - 所有数据均被切片封装为 PCIe 事务层数据包（TLP，Transaction Layer Packet）。
2. **往返时延（RTT）**：
    - PCIe 的典型往返延迟在 **1~2 微秒**。
    - **核心禁忌**：任何同步读取显存的操作（如 `cudaMemcpy(DeviceToHost)` 或在 PyTorch 中调用 `.item()`）都将引发 CPU 同步等待 PCIe TLP 完成包，彻底摧毁流水线吞吐。

3.2 内存映射 I/O (MMIO) 与 GPU Doorbell 敲铃机制

CPU 控制 GPU 的底层执行流程：

1. **PushBuffer 机制**：
    - 驱动程序在 Host 内存中分配一块环形缓冲区（PushBuffer），CPU 准备好待执行 Kernel 的执行上下文、Grid/Block 配置及参数指针并写入该缓冲区。
2. **Doorbell（敲门铃）**：
    - GPU 内部的控制寄存器通过 Base Address Register (BAR) 空间映射到 CPU 的物理地址空间（MMIO）。
    - CPU 只需向特定的 Doorbell 寄存器写入一个代表最新命令索引的 32/64 位数值（单次 PCIe 内存写事务），GPU 硬件接口立刻捕获，自主通过 DMA 从 PushBuffer 拉取指令包，实现极低开销的异步触发。

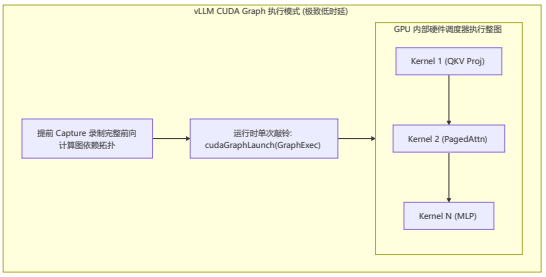
3.3 锁页内存（Pinned Host Memory）与异步 DMA 传输

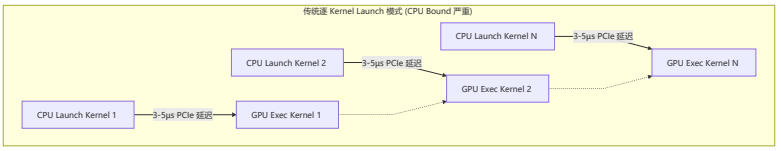
1. **虚拟内存与缺页风险**：
    - 操作系统可分页内存随时可能被 OS 交换至磁盘或重映射物理页，GPU DMA 控制器无法直接介入。
2. **Pinned 机制**：
    - vLLM 在初始化时通过 `cudaHostAlloc` 或 `torch.empty(..., pin_memory=True)` 锁定大块连续 Host 内存；
    - 在进行 KV Cache 换入换出或模型权重加载时，完全委托 GPU 的 **异步 DMA 拷贝引擎（Copy Engine）** 完成传输，CPU 在提交拷贝请求后立即返回，与 GPU 计算并发重叠。

3.4 CUDA Stream、Event 与 CUDA Graph 拓扑消除 Host 开销

大模型 Decode 阶段每个 Token 的计算耗时极短（例如在 H100 上，小 Batch 下可能仅需 1~3ms），而由数十个 Transformer Layer 组成的计算流程包含数百个底层算子。

- **逐算子 Launch 开销瓶颈**：每次 CPU Launch Kernel 都有 3~5μs 的软硬件驱动协议开销，累积后将达到几百微秒，直接导致 GPU 长期处于“吃不饱”的算力饥饿状态。



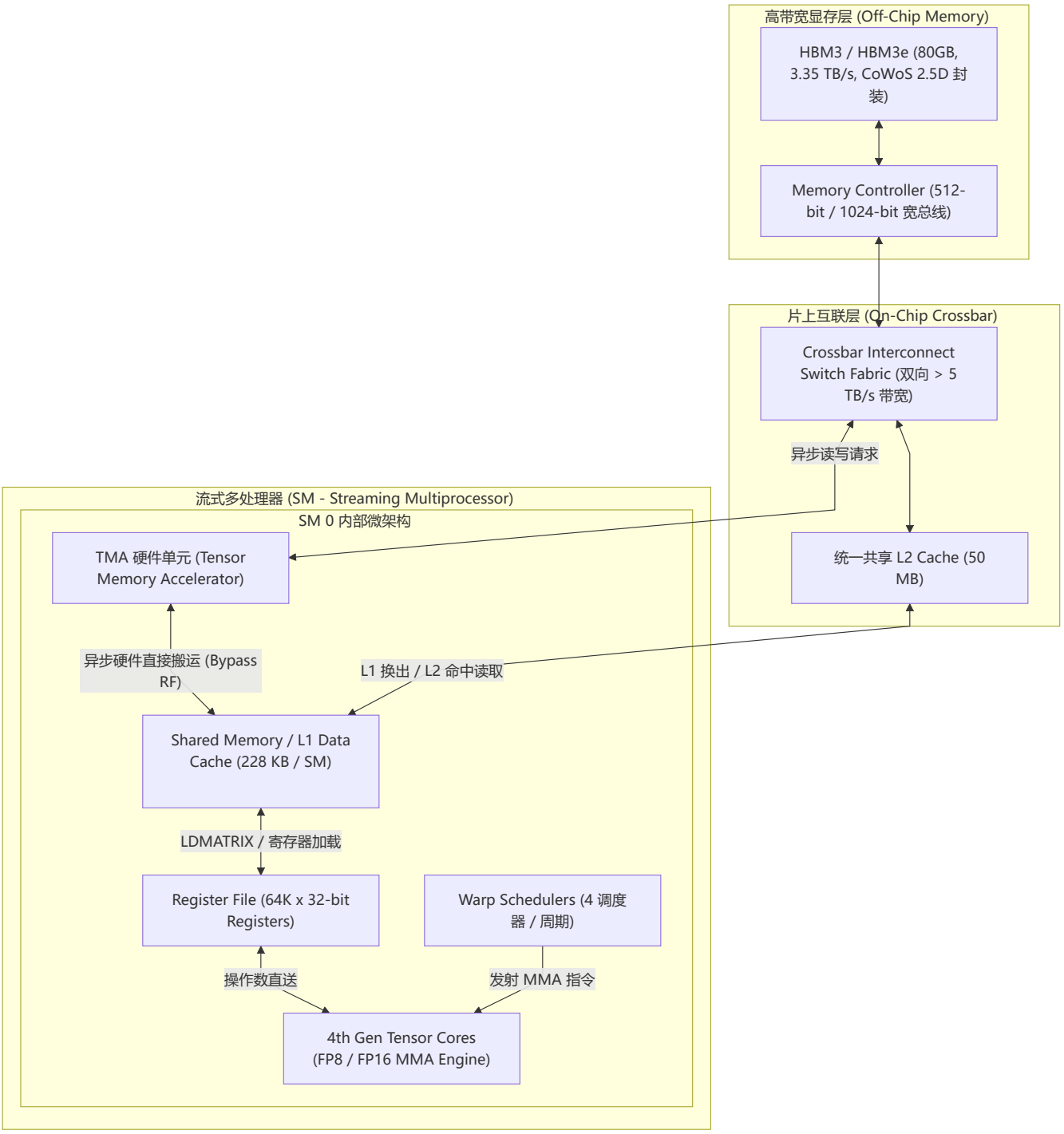


- **CUDA Graph 破局方案**：
    1. 在引擎预热（Warmup）阶段，针对不同 Batch Size 预先“录制”完整的静态算子执行拓扑，固化为硬件执行图；
    2. 运行时，CPU 发起一次 `cudaGraphLaunch` 单指令敲门铃，GPU 硬件工作流引擎全权自律执行整图；
    3. **动态 Block Table 注入**：vLLM 将 Block Table 数组分配在预先固定的显存地址上，每次 Decode 仅需覆写该固定数组的内容，无需重新捕获或重建计算图。

---

4. 单卡微架构层：GPU 内部硬件模块通信

在单个芯片内部（以 NVIDIA Hopper H100 为例），包含高达 132 个流式多处理器（SM）、数百兆片上 SRAM 与多达 80GB~141GB 的 HBM3/HBM3e。整个推理算子计算过程是一场极其惊人的片内微架构通信接力。



4.1 SM 内部：Warp 调度器、寄存器堆与 Tensor Core 间传输

1. **Warp 发射流水线**：
    - SM 包含 4 个处理子块（Sub-core），每个子块配备独立的指令缓存与 Warp Scheduler；
    - 调度器每个时钟周期调度就绪 Warp 发射指令到执行流水线。
2. **寄存器堆（Register File）通信**：
    - 提供整个芯片中最高的物理带宽（**> 20 TB/s 聚合带宽**），操作数在单个周期内直接馈入 FP16/FP8 Tensor Core 执行 MMA（Matrix Multiply-Accumulate）乘加运算。

4.2 SM 与 Shared Memory (SRAM) / L1 Cache 访存网络

1. **极速 SRAM 空间**：
    - 每个 SM 拥有高达 228 KB 的片上 SRAM，访问时延仅需 **20~30 个时钟周期（约几纳秒）**，聚合带宽超 **10 TB/s**。
2. **Bank 冲突（Bank Conflict）规避**：
    - SRAM 划分成 32 个 4 字节的独立 Bank。若同一个 Warp 内的多个线程访问同一 Bank 内的不同地址，硬件将强制排队串行化。高性能 Attention 内核通过地址异或混淆（Swizzling）确保并行满速加载。

4.3 异步拷贝与硬件加速器（NVIDIA TMA 异步张量传输）

- **传统方式的缺点**：数据从 HBM 载入 Shared Memory 必须先经过寄存器（Global Mem -> L2 -> Register -> Shared Mem），不仅消耗宝贵的寄存器空间，而且占用大量 SM 计算周期的指令发射槽。
- **Hopper TMA 革命**：
    1. TMA 硬件引擎是一个独立的硬件协处理器，具备硬件级多维张量坐标计算能力；
    2. 软件仅需下发一条指令描述张量维度，TMA 直接在 Global Memory 与 Shared Memory 之间建立 DMA 级硬直连；
    3. 配合 `cuda::barrier` 硬件异步栅障，让计算与数据预取完全解耦重叠。

4.4 Crossbar 片上互连网络与 L2 Cache 交互

- **片上纵横开关（Crossbar）**：
    - 连接所有 SM 与全局共享的 L2 Cache（通常 50MB 左右）。
    - Crossbar 必须提供足以支撑所有 SM 满负荷突发的横切带宽（Cross-sectional Bandwidth **> 5 TB/s**），负责处理高并发访存事务与缓存一致性广播。

4.5 L2 Cache 到 HBM3/HBM3e 的 CoWoS 宽总线访存通信

1. **物理层与先进封装**：
    - 采用台积电 **CoWoS（Chip-on-Wafer-on-Substrate）2.5D 硅中介层** 封装技术，逻辑计算芯片与 HBM3 颗粒紧密贴合。
    - 拥有多达 1024-bit 或更高的超宽总线接口，HBM3e 提供了高达 **3.35 TB/s** 的单卡显存吞吐。
2. **Decode 阶段的“内存墙”物理实质**：
    - 在自回归生成阶段，每一次生成 1 个 Token，都要从 HBM 中读取该请求累积的全部历史 KV Cache；
    - 计算强度（FLOPs/Byte）通常小于 1，硬件完全受制于从 HBM 经过 CoWoS 宽总线、Memory Controller 搬运数据到 L2 Cache 的通信极限。

4.6 PagedAttention 算子的微架构寻址与通信放大剖析

PagedAttention 在硬件层面上彻底打破了连续显存假设：

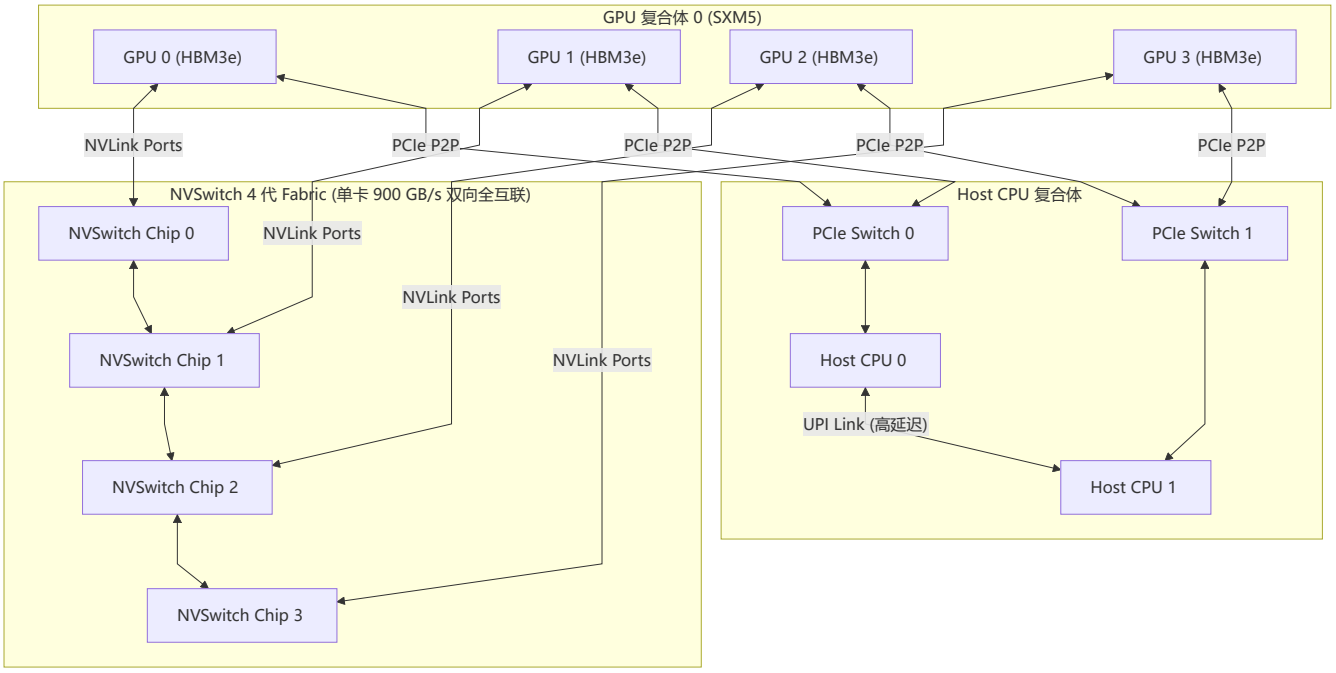
```
逻辑 Token 序列:
[ Token 0 ~ 15 ]    -->    [ Token 16 ~ 31 ]    -->    [ Token 32 ~ 47 ]
        |                          |                          |
   Block Table (常量显存/L2 Cache 映射查表)
        |                          |                          |
        v                          v                          v
物理 HBM 显存块:
[ 物理 Block #728 ]        [ 物理 Block #12 ]         [ 物理 Block #3491 ]
```

1. **寻址通信（Pointer Chasing）**：
   - 在计算当前 Query 与历史 Key 的点积时，CUDA 线程不能简单地使用连续基地址做线性偏移，而必须先查询由调度器维护的 **Block Table** 数组；
   - vLLM 通过将 Block Table 保存在常量显存（Constant Memory）或高命中率的 L2 Cache 中，使得每次物理块地址查找仅耗费几个周期的片上延迟。
2. **块尺寸设计权衡（Block Size Trade-off）**：
   - **Block Size = 16 或 32** 是通信效率的最佳平衡：
     - 若尺寸过小（如 4 或 8）：内存读取无法充分填满单次 128 字节的 Cache Line，总线突发传输效率急剧下跌；
     - 若尺寸过大（如 64 或 128）：动态并发场景下的显存尾部碎片膨胀，导致显存利用率降低。

---

5. 单机多卡层：Scale-Up 节点内硬件互连通信

针对单卡无法容纳的大模型（如 LLaMA-3-70B 需至少双卡或四卡运行），节点内多卡互联构成了 Scale-Up 扩展域。



5.1 PCIe P2P 与 PCIe 交换芯片（PLX）通信

1. **同 PLX 交换机通信**：
   - 在低成本消费级卡（如 RTX 4090）搭建的节点中，若 GPU 均挂接在同一块 PLX PCIe Switch 之下，

TLP 数据包可在 Switch 内部完成硬件线速回环转发，无需经过 CPU 根复合体（Root Complex）。

2. **跨 Socket 的性能断崖**：
    - 若两块 GPU 分属不同的 CPU Socket，P2P 通信必须跨越 Intel UPI 或 AMD xGMI 总线。时延由数百纳秒骤增至微秒级，带宽腰斩，直接破坏多卡张量并行性能。

5.2 NVLink 与 NVSwitch 全互联拓扑

1. **NVLink 物理特性**：
    - **NVLink 4（H100/H800）**：提供 18 条高速链路，单卡双向带宽高达 **900 GB/s**（约为 PCIe 5.0 的 14 倍）。
    - **NVLink 5（B200）**：进一步翻倍至 **1800 GB/s (1.8 TB/s)**。
2. **NVSwitch 架构消除拓扑偏差**：
    - 8 卡服务器内部集成独立 NVSwitch 物理芯片，构建了一套全对称、无阻塞的胖树网络；
    - 节点内任意两块 GPU 之间通信均只需经历单跳（Single-hop），彻底摆脱了传统多卡拓扑在 Ring 环形结构上存在的跨步等待与跳数不均瓶颈。

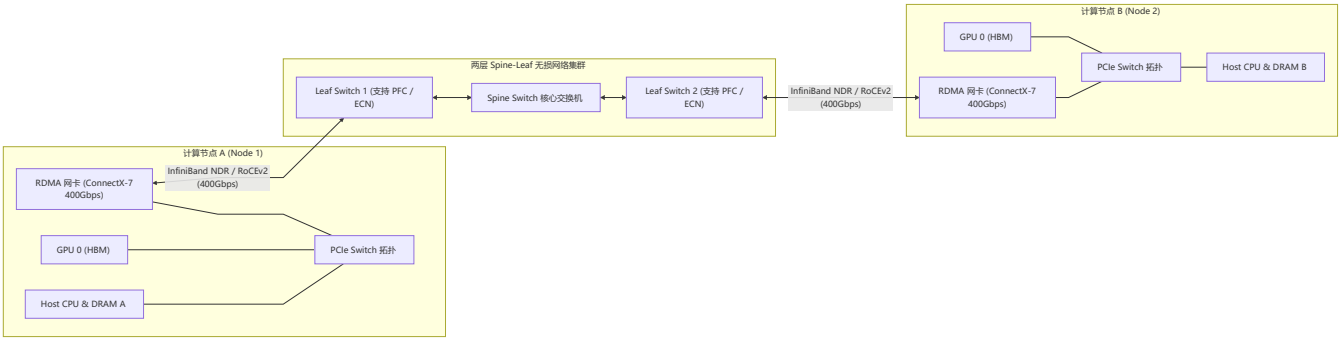
5.3 GPUDirect P2P 与 CUDA IPC 共享显存句柄

1. **跨进程物理显存映射**：
    - vLLM 默认多进程架构下，每个 Rank Worker 是独立的 Linux 进程。
    - Worker 0 通过调用 `cudaIpcGetMemHandle`，将本地一段显存（如权重或激活值）转换为一个 64 字节的全局不透明句柄；
    - 通过 Unix Domain Socket 发送至其他 Worker 进程；
    - 对端调用 `cudaIpcOpenMemHandle` 将其直接映射到自身的虚拟地址空间中；
2. **硬件级直通执行**：
    - 后续读写指令在 GPU 硬件层直接转换为跨卡 NVLink 内存总线事务，完全不需要经过 CPU 操作系统的任何介入。

---

6. 跨机集群层：Scale-Out 节点间硬件互连通信

当参数量达到超大规模（如 405B 模型，单机 8 卡显存不足），或在构建大规模 Prefill-Decode 分离集群时，通信必须扩展到网络集群尺度（Scale-Out）。



6.1 网络基础设施：InfiniBand 与 RoCE v2 无损以太网

1. **InfiniBand (IB)**：
    - 基于专有信用（Credit-Based）流控协议，实现硬件级的绝对无丢包，端口速度高达 NDR 400Gbps，端到端延迟 **< 1.5 微秒。**
2. **RoCE v2 (RDMA over Converged Ethernet)**：
    - 在标准以太网上封装 UDP-RDMA 报文；
    - 依赖网络交换机开启 **PFC（优先级流控）** 与 **ECN（显式拥塞通知）** 构建无损以太网，避免因拥塞丢包引发的 TCP 超时重传风暴。

6.2 GPUDirect RDMA (GDR) 零拷贝技术

GPUDirect RDMA（GDR）是现代大模型跨节点分布式扩展（如跨机 TP/PP/EP 以及 Prefill-Decode 分离）的核心使能技术。它彻底打破了传统网络通信中必须由 Host CPU 内存充当“数据中转站”的架构壁垒。



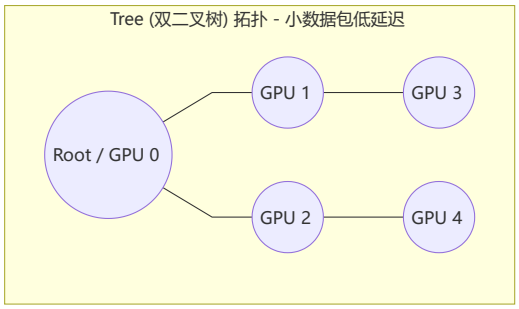


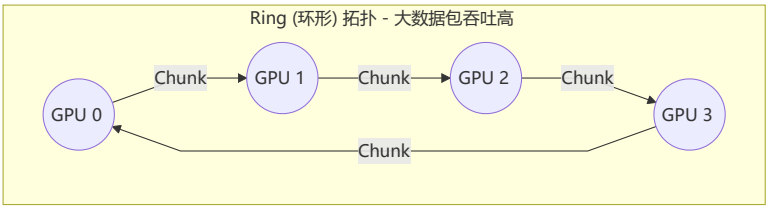
关键机制剖析：

1. **通信路径革新对比**：
    - **传统非 GDR 路径（7 步链路 / 4 次数据搬运）**：
      GPU 显存与网卡无法直接寻址彼此。数据必须由 GPU 先通过 PCIe 搬运至 Host Pinned Memory，CPU 再将其复制到内核网络套接字缓冲区，网卡发起 DMA 读取后再发送至网络。接收端同样经历多级中转，导致 PCIe 总线被重复占用（单次跨机传输占用 4 次 PCIe 链路），同时消耗宝贵的 CPU 算力与 Host 内存带宽。
    - **GPUDirect RDMA 直通路径（零 CPU 拷贝）**：
      通过驱动层加载 `nvidia-peermem` 内核模块，Mellanox OFED 驱动与 NVIDIA CUDA 驱动实现底层物理页表互通。网卡 HCA 的 PCIe 控制器与 GPU 显存控制器通过同一级 **PCIe Switch（或 Root Complex P2P）** 建立 Peer-to-Peer DMA 通道。网卡直接向 GPU 的物理显存地址读取或写入数据包，**完全跳过 Host 操作系统、CPU 协议栈和系统主存。**
2. **性能与时延飞跃**：
    - **端到端单向延迟**：从传统路径的 20~50μs 骤降至 **2~4μs。**
    - **CPU 卸载（Full Offload）**：大吞吐传输（如 Prefill 产生的大量 KV Cache 跨机迁移）期间，Host CPU 占用率为 **0%。**
    - **PCIe 总线减负**：消除了对 Host DRAM 的往返冗余拷贝，PCIe 有效吞吐翻倍。

6.3 NCCL 集合通信原语的底层拓扑与算法（Ring vs Tree vs NVLS）

NVIDIA NCCL（集合通信库）作为 PyTorch 分布式底层驱动，在大模型分布式推理中支撑核心通信：





1. **Ring AllReduce**：
    - 节点形成闭环，每个节点发送 `2(N - 1)` 次分片。带宽利用率接近 100%，极适合长序列 Prefill 阶段传输大激活张量；但通信延迟与节点数 `N` 呈线性关系。
2. **Double Binary Tree AllReduce**：
    - 建立两棵不相交的逻辑二叉树，通信步长降低至 `O(log N)`。在 Decode 阶段单步生成仅需同步小张量（如单个 Token 的 Embedding）时，树状算法延迟显著优于环形算法。
3. **NVLS (NVLink SHARP)**：
    - 依赖最新 NVSwitch 硬件中的网内计算引擎，在交换机内部瞬间完成向量加法聚合并广播回各卡，消除一半传输量。

6.4 网内计算硬件加速（In-Network Computing / SHARP）

跨机交换机通过 Mellanox SHARP 硬件模块，将 AllReduce 算子下沉到网络交换机物理芯片执行：

- 节点将数据推向交换机；
- 交换机芯片线速执行定点或浮点加法聚合；
- 交换机直接分发最终张量给所有机器，彻底避免主机多次转发造成的网络骨干带宽折损。

---

7. 分布式并行策略与通信模式剖析

在实际部署中，vLLM 根据模型结构与硬件环境组合不同的并行策略：



7.1 张量并行（Tensor Parallelism, TP）通信时机与开销

张量并行将每一层的权重矩阵拆分到多张 GPU 上并行计算：

1. **Self-Attention 模块**：
    - QKV Projection 采用列并行，各卡计算局部的 Head，无需通信；
    - Attention 输出执行 Row Parallel 矩阵乘后，各卡得到局部输出，**必须执行一次全局 All-Reduce 同步。**
2. **MLP 模块**：
    - Gate / Up Projection 列并行；
    - Down Projection 行并行，计算后**必须执行第二次全局 All-Reduce 同步。**
3. **通信对延迟的致命影响**：

- 对于一个拥有 80 层的 Transformer 模型，**生成单个 Token 必须在关键路径上串行穿透 160 次 All-Reduce 集合通信**！
- 这意味着只要通信延迟增加 10μs，单个 Token 的生成延迟就将增加 1.6ms。因此 TP 强绑定于低延迟高带宽的 NVLink 节点内。

7.2 流水线并行（Pipeline Parallelism, PP）点对点通信

1. **层级切分与传输**：
   - 将模型按 Layer 划分到不同的机器或 GPU 阶段（Stage）；
   - 相邻 Stage 之间仅需传输 Hidden States 激活张量，原语为轻量级的异步 `ISend` / `IRecv`。
2. **优缺点分析**：
   - 优点：通信频次极低（每层仅一次），可部署在跨机低速网络；
   - 缺点：自回归解码时微批次填充困难，极易产生巨大的流水线计算空泡（Bubble）。

7.3 专家并行（Expert Parallelism, EP / MoE）的 All-to-All 路由通信

针对 Mixtral、DeepSeek 等稀疏专家模型（MoE）：

1. **两阶段 All-to-All 核心链路**：
   - **Dispatch 通信**：Gating 网络算出 Token 应归属的目标专家卡，通过全局 **All-to-All** 将 Token 数据包分发到各卡对应的 Expert；
   - **Combine 通信**：Expert 计算完成后，再次通过全局 **All-to-All** 将专家输出数据原路传回源 GPU，并按 Gating 权重执行加权求和。
2. **热点倾斜与通信重叠**：
   - 当多个 Token 均路由到同一个热门专家时，会导致严重的通信网络拥塞与计算负载不均衡。vLLM 引入了通信与计算重叠（Overlap）技术，在等待部分专家传输的同时，抢先计算已到达的数据。

7.4 序列并行（Sequence Parallelism, SP / Context Parallelism, CP）通信

针对 128K~1M 超长序列上下文：

1. **Ring-Attention 环形通信**：
   - 将超长 Prompt 沿 Sequence 维度切分到不同 GPU；
   - 在计算自注意力时，各 GPU 将自身的 Key/Value 块通过环形拓扑向下一个 GPU 传递；
   - 边接收对端的 KV，边异步计算本卡 Query 与接收到的 KV 的注意力得分，实现通信掩盖。

7.5 投机推理（Speculative Decoding）Draft-Target 协同通信

1. **投机草稿生成与验证**：
   - **草稿阶段**：小模型（Draft Model）快速生成 `K` 个候选 Token；
   - **验证阶段**：大模型（Target Model）执行一次并行前向传播，通过掩码（Mask）机制同时校验该 `K` 个候选 Token 的接收概率。
2. **协同通信拓扑**：
   - 同实例共存：Draft 与 Target 在同一进程共享显存指针，通过内部张量切片直接交换候选 Token，通信

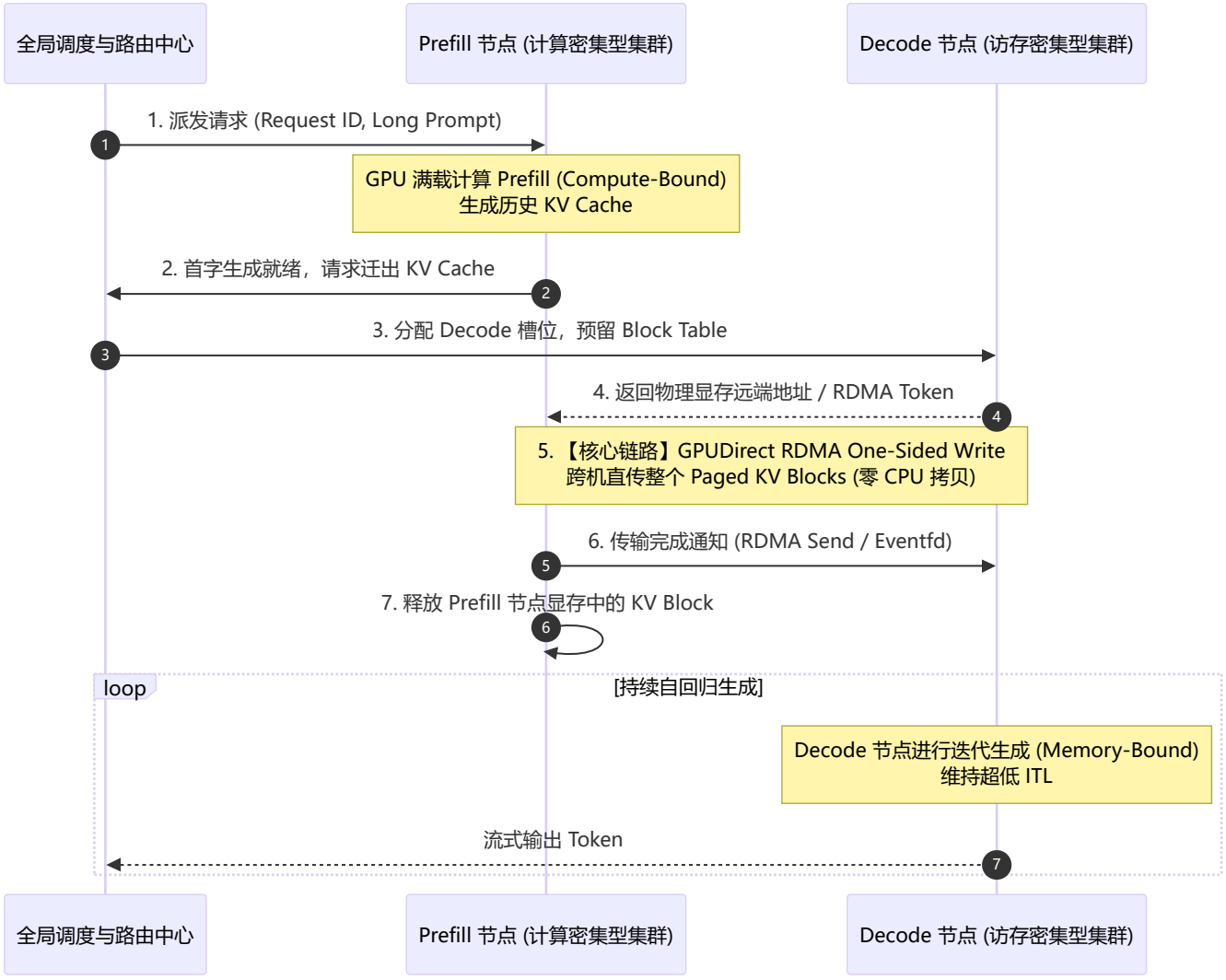
开销为 0；

- 分布式解耦实例：Draft 引擎与 Target 引擎通过专用 IPC / gRPC 传输验证树（Tree Attention），减少验证步数。

---

8. Prefill 与 Decode 解耦架构（PD 分离）跨节点通信

“**Prefill-Decode 解耦（Disaggregated Prefill and Decode）**”是 2024~2026 年大模型工业界最关键的系统级演进（如 Mooncake、DistServe、Splitwise 以及 vLLM 官方架构探索）。



8.1 PD 分离的核心痛点与架构演进

1. **计算特征的天然冲突**：
   - **Prefill 阶段**：高并行、计算密集（Compute-Bound），单次计算吞吐极大，需要打满 Tensor Core；
   - **Decode 阶段**：单 Token 依赖历史状态、访存密集（Memory-Bound），依赖超高 HBM 带宽，需要维持极低且平稳的 ITL。
2. **传统混布的干扰**：
   - 当一个长 Prompt 的 Prefill 突发插入正在执行的 Decode 批次中，当前批次所有 Decode 请求的生成延迟将被严重拉长，造成剧烈抖动与 P99 延迟恶化。
3. **架构解耦**：

- 将集群物理拆分为 Prefill 计算集群与 Decode 生成集群，各司其职，达到最佳硬件能效比。

8.2 KV Cache 跨机远端传输协议与 GDR 写入机制

PD 分离的核心生命线是：**KV Cache 跨机迁移时间必须显著小于在 Decode 节点上重新 Prefill 的时间。**

1. **RDMA One-Sided Write（单边写直传）**：
   - Decode 节点通过元数据信道将分配好的 Paged 物理显存地址与 Memory Key 告知 Prefill 节点；
   - Prefill 节点的网卡使用 **RDMA Write With Immediate** 协议，通过 GPUDirect RDMA 直接将庞大的 KV Cache 块远端写入 Decode 节点的 HBM 显存段；
   - 整个过程 Decode 节点的 CPU 完全不参与数据搬运，实现接近物理网卡理论上限的满速（400Gbps = 50 GB/s）低延迟灌入。
2. **层间流水线传输掩盖（Pipelined KV Streaming）**：
   - 在 Prefill 计算模型深层 Transformer 时，浅层已经计算完毕的 KV Cache 可以立刻触发异步 RDMA 传输，使网络传输与模型后半部分的计算完全重叠（Overlap）。

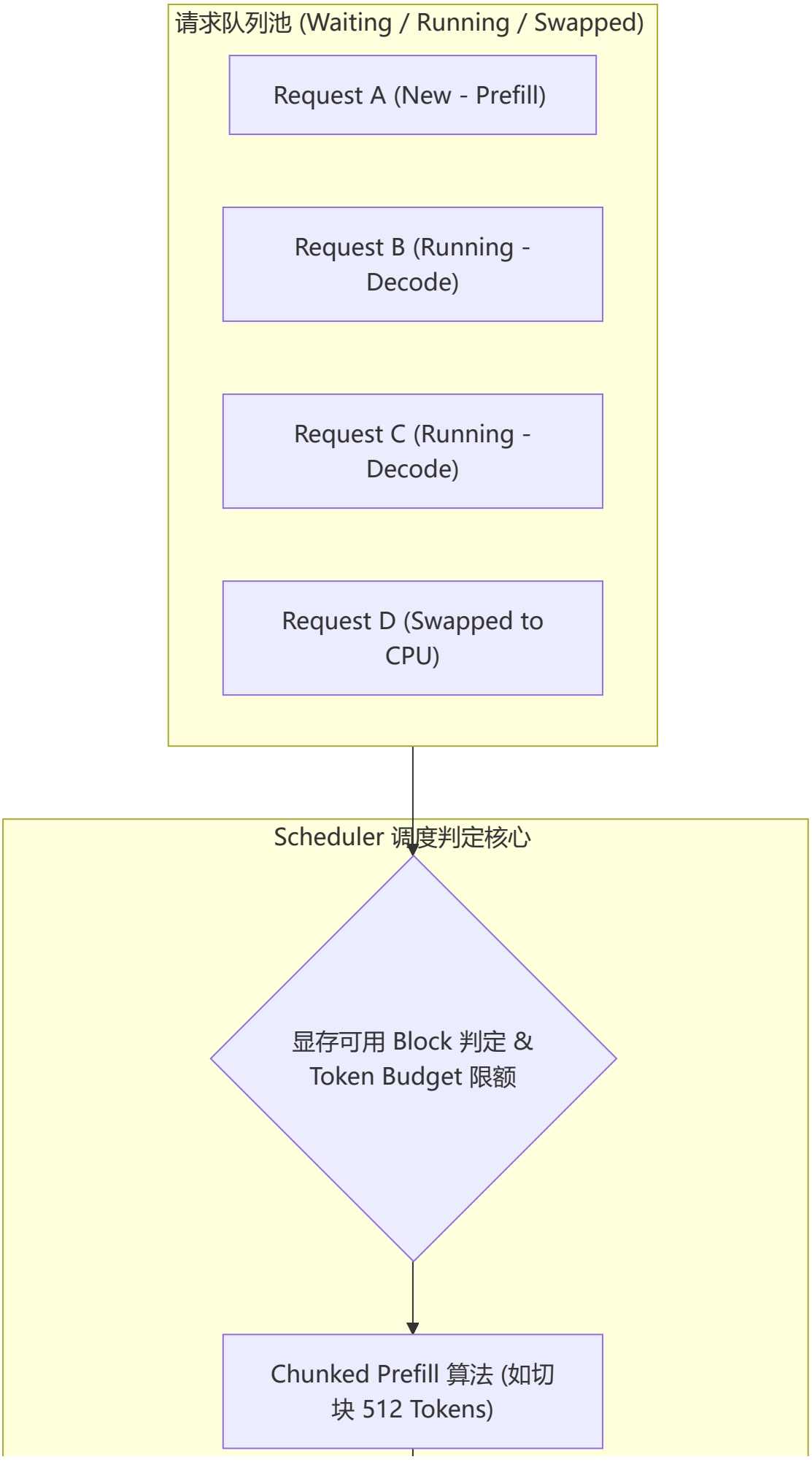
8.3 元数据协同（Sequence ID、Block Table 映射与流控）

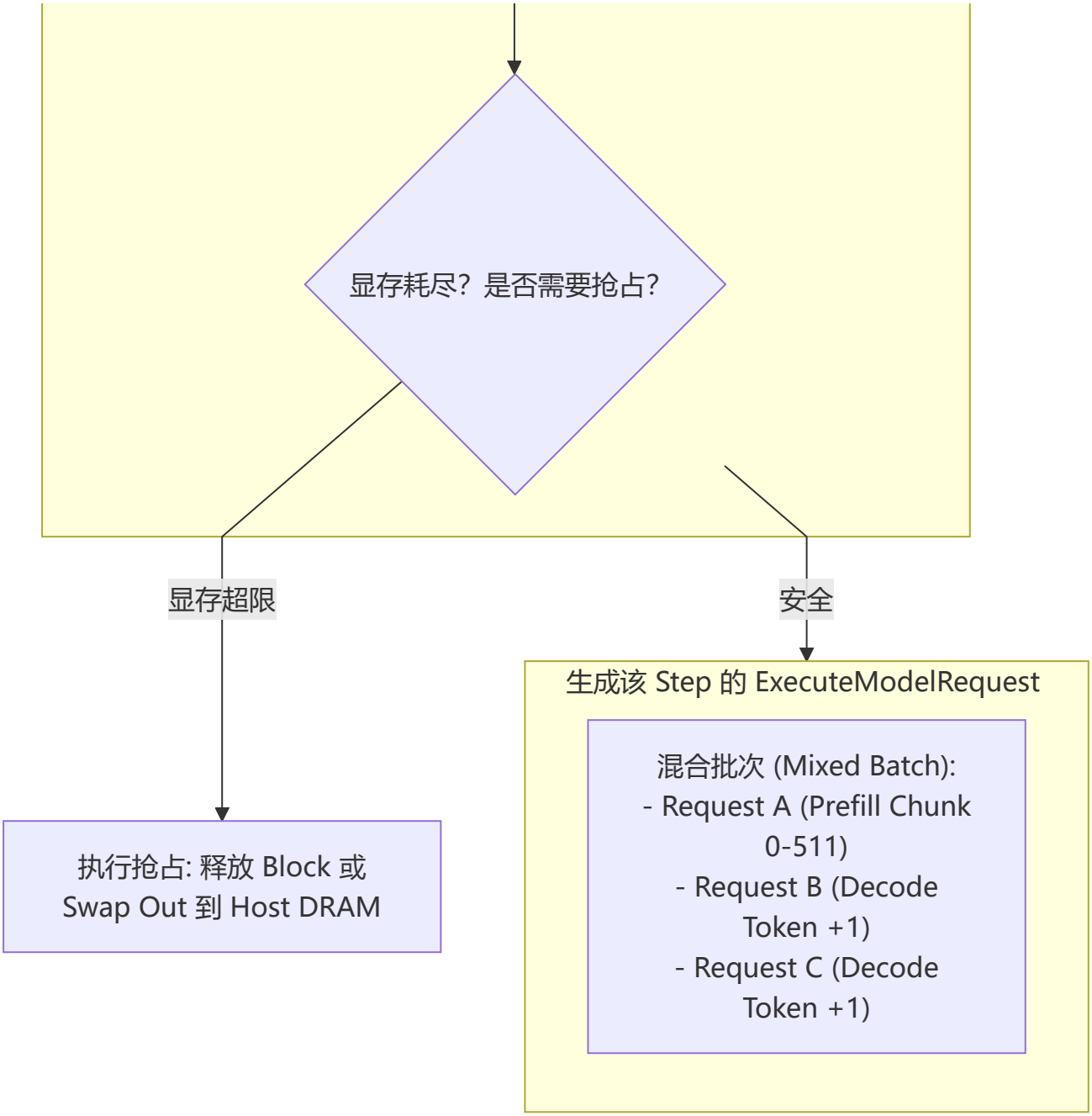
- **状态同步机制**：
  - 两端各自的 `BlockSpaceManager` 通过轻量级的 gRPC / ZMQ 元数据通道同步 Sequence 状态、采样参数与物理 Block Table 映射表；
  - 配套完善的流控与超时重传逻辑，保证在分布式异常状态下显存块的安全回收。

---

9. 统一调度方案与迭代管理机制

调度器是大模型推理引擎的大脑中枢，决定系统以何种节拍向前演进。


请求队列池 (Waiting / Running / Swapped)
Request A (New - Prefill)
Request B (Running - Decode)
Request C (Running - Decode)
Request D (Swapped to CPU)
Scheduler 调度判定核心
显存可用 Block 判定 & Token Budget 限额
Chunked Prefill 算法 (如切块 512 Tokens)



9.1 连续批处理（Continuous Batching / Iteration-level Scheduling）

1. **迭代级粒度调度（Iteration-level Scheduling）**：
    - 彻底摒弃传统以完整请求为粒度的静态 Batching；
    - 调度器在每个 Token 生成的极短时钟周期内均重新做一次调度决策：
        - 生成完成（`<eos>` 或达到最大长度）的请求立刻离队，物理 Block 毫秒级回收；
        - 等待队列中的新请求立即插入空缺位置开始 Prefill；
        - 持续运行的请求分配下一个物理显存槽位继续 Decode。

9.2 分块预填充（Chunked Prefill）与时延平准通信调度

1. **超长上下文的算力垄断风险**：
    - 若某请求输入长达 32K Token，传统 Prefill 会霸占整机几百毫秒，导致其他已有会话严重卡顿。

2. **Chunking 机制**：
    - 设定全局 Token 吞吐预算（如 `max_num_batched_tokens = 2048`）；
    - 将长序列拆解为固定步长（如 512 或 1024 Token）的分块；
    - 将 Prefill 分块与正在进行的 Decode 请求打包进**同一个混合 Batch（Mixed Batch）**；
    - 从而将长序列 Prefill 的计算与通信开销均匀分摊至多个迭代中，兼顾系统的吞吐量与单 Token 延迟稳定性。

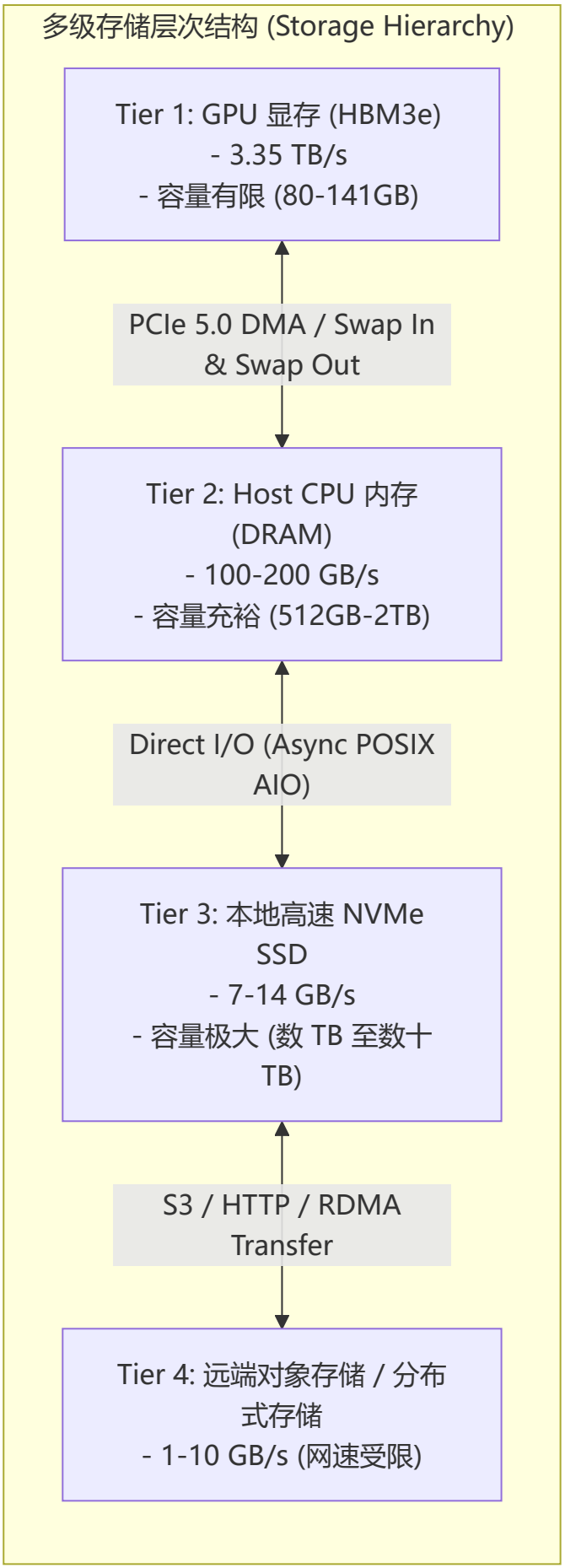
9.3 调度决策闭环与跨模块反馈时延

- **微秒级决策要求**：
    - 调度器每次判定耗时必须压缩在 **< 100 微秒** 以内，否则调度决策时间会显著反噬单 Token 解码速度；
    - 显存状态监控数据结构采用位图（Bitmap）和高效率 C++ 结构体维护，规避深度递归与冗余查表。

---

10. 模型与显存动态加载、卸载与换入换出策略

推理系统的内存管理是决定吞吐上限与系统稳定性的关键工程。


多级存储层次结构 (Storage Hierarchy)
Tier 1: GPU 显存 (HBM3e)
- 3.35 TB/s
- 容量有限 (80-141GB)
PCIe 5.0 DMA / Swap In & Swap Out
Tier 2: Host CPU 内存 (DRAM)
- 100-200 GB/s
- 容量充裕 (512GB-2TB)
Direct I/O (Async POSIX AIO)
Tier 3: 本地高速 NVMe SSD
- 7-14 GB/s
- 容量极大 (数 TB 至数十 TB)
S3 / HTTP / RDMA Transfer
Tier 4: 远端对象存储 / 分布式存储
- 1-10 GB/s (网速受限)

10.1 模型权重启动加载与并行广播分发

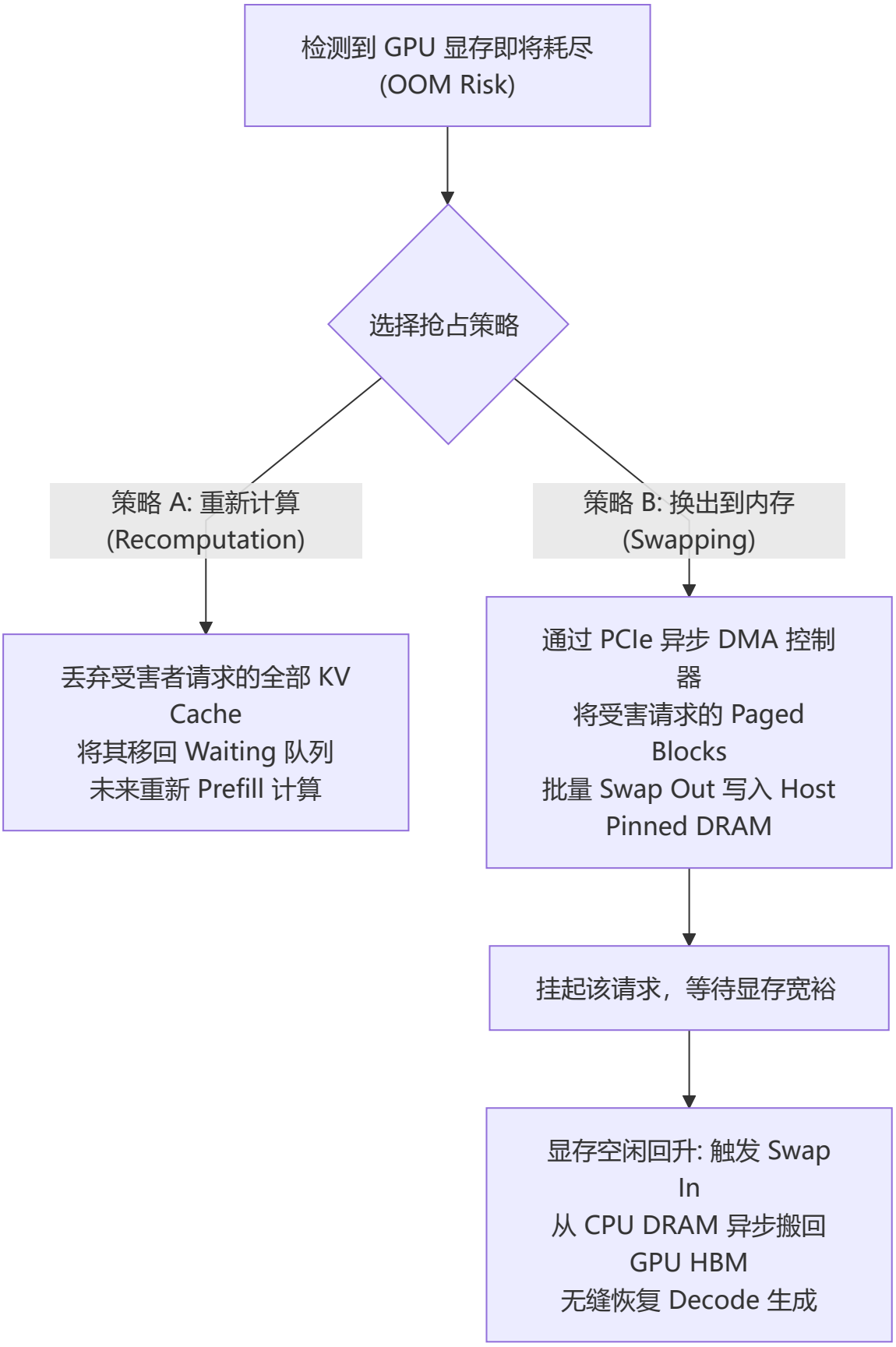
1. **Safetensors 与 mmap 零拷贝加载**：
    - 使用内存映射文件（`mmap`），操作系统仅建立虚拟地址映射，真正读取数据发生在实际访存触发缺页之时，大幅缩短加载初始化时延。
2. **多卡并发读取 vs 广播**：
    - **NCCL Broadcast**：由 Rank 0 负责从磁盘单点读取权重，并通过节点内 900 GB/s 的 NVLink 以广播形式分发给各卡，避免多进程同时读取磁盘导致 I/O 吞吐崩溃；
    - **Direct I/O 并行读**：在高端 NVMe RAID 环境下，各 Worker 自行读取属于自身 TP 分片的特定偏移区间，最大化磁盘并行吞吐。

10.2 动态多租户 LoRA 适配器热插拔与层级加载

1. **统一批处理（S-LoRA / Punica）**：
    - 基础大模型权重全卡共享，数百个 LoRA 权重动态附加；
    - 利用 `sgmv`（Segmented Gather Matrix-Vector）内核在单次前向传播中针对不同 Token 调用不同的 LoRA 权重。
2. **多级缓存调度**：
    - GPU 显存中开辟固定的 LoRA 显存池；
    - 基于 LRU 淘汰算法，热点 LoRA 常驻 GPU HBM，次频 LoRA 缓存在 Host CPU 内存，冷门 LoRA 异步预取自本地 NVMe SSD，实现服务零停机的无缝动态热插拔。

10.3 KV Cache 显存换入换出（Swapping）与抢占（Preemption）机制

在高并发突发流量下，当 GPU 显存耗尽时，调度器启动抢占防护策略：



1. **Recomputation（重新计算）**：
    - 适用于输入短、算力损耗低的请求。直接丢弃已生成的 KV Cache，等待显存恢复后重新做一次 Prefill。
2. **Swapping（换出到 Host 内存）**：

- 适用于超长序列（如已有上万 Token 上下文）的请求；
- `BlockSpaceManager` 通过 PCIe 异步 DMA 将该请求的全部物理 Block 批量搬迁至 Host 锁页内存中（Swap Out）；
- 待显存水位下降，再异步调度 DMA 搬回 GPU（Swap In），精确恢复上下文，保护宝贵的首字计算收益。

10.4 分级存储卸载架构（HBM -> Host DRAM -> Local NVMe -> Remote Object Store）

先进的大规模长文本缓存架构（如 **LMCache**、Mooncake 存储池）：

1. **Tier 1 (GPU HBM)**：纳秒级延迟，3.35 TB/s 极限带宽，支撑当前迭代执行；
2. **Tier 2 (Host DRAM)**：微秒级延迟，百 GB/s 带宽，充当弹性换入换出缓冲池；
3. **Tier 3 (Local NVMe)**：毫秒级延迟，利用 Linux 用户态存储开发框架（SPDK）绕过操作系统文件系统，直接将持久化 KV Cache 写入裸固态硬盘；
4. **Tier 4 (Remote Object Store / Redis)**：跨会话长周期沉淀历史会话或系统 Prompt 前缀，支持百万 Token 级会话的秒级前缀命中复用（Prefix Caching）。

---

11. 软硬件通信层级性能矩阵与系统级优化全景

下表总结了从芯片内部微架构到跨机分布式网络的完整通信链路关键物理参数与优化手段：

通信层级	物理载体 / 协议栈	典型单向带宽	典型往返时延 (RTT)	vLLM 核心传输内容	工业界核心优化技术
SM 内部寄存器	Register File (RF)	20 ~ 30 TB/s (聚合)	< 1 ns (~几周期)	矩阵乘中间累加值、向量操作数	寄存器压力控制、循环展开、指令重排
片上共享内存	Shared Memory / L1 Data Cache	10 ~ 15 TB/s	~5-10 ns (20-30 周期)	FlashAttention / PagedAttn 局部计算块	TMA 异步搬运、Swizzle 消除 Bank 冲突
GPU 片上互联	Crossbar Interconnect -> L2	4 ~ 6 TB/s	~20-30 ns (数十周期)	L2 命中数据、跨 SM 广播数据	访存合并（Coalescing）、数据局部性优化
全局高带宽显存	CoWoS 2.5D 总线 -> HBM3/HBM3e	2.0 ~ 3.35 TB/s	50 ~ 100 ns (数百周期)	基础模型权重、全量 Paged KV Cache	PagedAttention 块对齐、FP8/FP4 KV 量化
单卡 Host 交互	PCIe 5.0 x16 / CXL 2.0	32 GB/s (双向 64 GB/s)	1.0 ~ 2.0 μs	批次元数据、KV Swap 换入换出	Pinned Memory 异步 DMA、CUDA Graph 消除 Launch

通信层级	物理载体 / 协议栈	典型单向带宽	典型往返时延 (RTT)	vLLM 核心传输内容	工业界核心优化技术
单机多卡互联	NVLink 4/5 + NVSwitch	450 ~ 900 GB/s	100 ~ 200 ns	张量并行 (TP) All-Reduce、MoE EP 路由	NVLS (硬件网内聚合)、CUDA IPC 零拷贝显存共享
跨机网络互联	InfiniBand NDR (400G) / RoCEv2	50 GB/s (400 Gbps)	1.5 ~ 3.0 μs	跨节点 TP/PP/EP 集合通信、PD 分离 KV 直传	GPUDirect RDMA (GDR)、SHARP 网内计算、PFC/ECN
进程间软件通信	POSIX 共享内存 / ZeroMQ / Ray	10 ~ 20 GB/s (内存带宽)	5 ~ 20 μs	RequestOutput、调度指令包、Token IDs	C++ 广播环形队列 (Broadcast RingBuffer)、零反序列化
应用层网络通信	HTTP/2, SSE, gRPC, WebSocket	1 ~ 10 Gbps (以太网)	10 ~ 50 ms (公网/局域网)	客户端 Prompt 输入、流式生成的文本 Chunk	异步增量 Detokenizer、连接多路复用、流式压缩

---

12. 总结与未来趋势展望

大语言模型的高性能推理绝非单纯的算力堆砌，而是一套**深度耦合了软件分布式调度系统、操作系统驱动接口、高速总线网络以及纳米级微电子互联**的系统通信工程。以 vLLM 为代表的优秀引擎正是精准把控了每一个层级的物理通信边界：

1. **软件栈**：通过异步事件流、ZeroMQ 解耦核心与共享内存零拷贝消除 CPU 与 Python 开销；
2. **接口栈**：通过 CUDA Graph 固化算子拓扑、Pinned DMA 卸载规避 PCIe 启动与传输等待；
3. **微架构栈**：通过 PagedAttention 与 TMA 硬件直连，使非连续存储适应 HBM 与 SRAM 的突发传输特性；
4. **扩展栈**：通过 NVLink 与 GPUDirect RDMA 配合 NCCL 树形与网内计算算法，化解了多卡张量并行（TP）与专家并行（EP）的集合通信延迟墙；
5. **架构栈**：通过 **PD 解耦分离（Disaggregated Prefill-Decode）** 与 **分级存储（Hierarchical Offloading）**，彻底打破了单卡资源边界与长文本时延抖动瓶颈。

在未来的硬件与算法演进中，**CXL 统一池化内存**、**硅光（Optical I/O）片上光互联**、**晶圆级超大芯片系统** 以及软件层面的 **全局分布式 KV 交换底盘（如 Mooncake / LMCache）** 将持续推进，最终构建出"**计算与通信全面融合、全链路硬件直通零拷贝**"的下一代 AI 原生基础设施体系。